```
                                ThreadContext.put(IP, header.getSys_comm().getTr_trm_ipadr());
                                ThreadContext.put(USER_ID, header.getSys_comm().getOptr_eno());

                                ServiceContext serviceContext = new ServiceContext(applicationName,
guid, active, requestHeaders, request, response, header);
                                ServiceContextHolder.setInstance(serviceContext);

                                // 서블릿 호출
                                filterChain.doFilter(wrapper, servletResponse);

                        }
                } catch (Exception e) {
                        log.error("Filter 처리 중 Exception 발생: {}", e.getMessage(), e);
                        response.sendError(HttpServletResponse.SC_BAD_REQUEST, "Filter 처리 중
Exception 발생.");
                        return;
                } finally {
                        if (ServiceContextHolder.getInstance() != null) {
                                ServiceContextHolder.removeInstance();
                        }
                        if (!ThreadContext.isEmpty()) {
                                ThreadContext.clearAll();
                        }

                }
        }

        private <T> T convertMapToDto(ObjectMapper mapper, Map<String, Object> map, Class<T> clazz) {
                map.remove("dtoLogicalName");
                return mapper.convertValue(map, clazz);
        }

}
package nhnis.fw.commons.filter;

import java.io.BufferedReader;
import java.io.ByteArrayInputStream;
import java.io.IOException;
import java.io.InputStreamReader;
import java.nio.charset.StandardCharsets;

import jakarta.servlet.ReadListener;
import jakarta.servlet.ServletInputStream;
import jakarta.servlet.http.HttpServletRequest;
import jakarta.servlet.http.HttpServletRequestWrapper;

public class CachedBodyHttpServletRequest extends HttpServletRequestWrapper {

        private final byte[] cachedBody;

        public CachedBodyHttpServletRequest(HttpServletRequest request) throws IOException {
                super(request);
                this.cachedBody = request.getInputStream().readAllBytes();
        }

        public byte[] getCachedBody() {
                return cachedBody;
        }

        @Override
        public ServletInputStream getInputStream() {
                ByteArrayInputStream inputStream = new ByteArrayInputStream(cachedBody);
```

```
                return new ServletInputStream() {

                        @Override
                        public boolean isFinished() {
                                return inputStream.available() == 0;
                        }

                        @Override
                        public boolean isReady() {
                                return true;
                        }

                        @Override
                        public void setReadListener(ReadListener arg0) {
                        }

                        @Override
                        public int read() throws IOException {
                                return inputStream.read();
                        }

                };
        }

        @Override
        public BufferedReader getReader() throws IOException {
                return new BufferedReader(new InputStreamReader(getInputStream(),
StandardCharsets.UTF_8));
        }

}
package nhnis.fw.commons.fos;

import java.io.IOException;
import java.io.InputStream;
import java.util.concurrent.TimeUnit;

import org.apache.hc.client5.http.classic.methods.HttpDelete;
import org.apache.hc.client5.http.classic.methods.HttpGet;
import org.apache.hc.client5.http.classic.methods.HttpPut;
import org.apache.hc.client5.http.config.RequestConfig;
import org.apache.hc.client5.http.impl.classic.CloseableHttpClient;
import org.apache.hc.client5.http.impl.classic.HttpClients;
import org.apache.hc.core5.http.ClassicHttpRequest;
import org.apache.hc.core5.http.ContentType;
import org.apache.hc.core5.http.io.HttpClientResponseHandler;
import org.apache.hc.core5.http.io.entity.EntityUtils;
import org.apache.hc.core5.http.io.entity.InputStreamEntity;
import org.slf4j.Logger;
import org.slf4j.LoggerFactory;
import org.springframework.beans.factory.annotation.Value;
import org.springframework.stereotype.Component;
import org.springframework.util.StringUtils;

import nhnis.fw.commons.context.SpringContext;
import nhnis.fw.commons.fos.enums.NameSpace;
import nhnis.fw.commons.fos.enums.Tenant;

@Component
public class ObjectStorageHandler {

        private static final Logger LOGGER = LoggerFactory.getLogger(ObjectStorageHandler.class);
```

```
    @Value("${storage.fos.connect-timeout:-1}")
    private int connectTimeout;

    @Value("${storage.fos.response-timeout:-1}")
    private int responseTimeout;

    private static final String DOT = ".";
    private static final String SLASH = "/";
    private static final String AUTHORIZATION = "Authorization";

    public ObjectStorageDto put(Tenant tenant, NameSpace nameSpace, String filePath, InputStream inputStream, ContentType contentType) throws IOException {
        String url = getAddress(tenant, nameSpace, filePath);
        HttpPut request = new HttpPut(url);
        setAuthToken(nameSpace, tenant, (ClassicHttpRequest)request);

        if (contentType == null) {
            contentType = ContentType.APPLICATION_OCTET_STREAM;
        }

        request.setEntity(new InputStreamEntity(inputStream, contentType));
        RequestConfig requestConfig = getRequestConfig();
        request.setConfig(requestConfig);

        try (CloseableHttpClient client = HttpClients.createDefault()) {
            HttpClientResponseHandler<ObjectStorageDto> responseHandler = response -> {
                int statusCode = response.getCode();

                if (statusCode >= 300) {
                    String body = response.getEntity() == null ? "" : EntityUtils.toString(response.getEntity());
                    throw new IOException("(PUT) Failed. status=" + statusCode + " body=" + body);
                }

                ObjectStorageDto dto = new ObjectStorageDto();

                if (response.getEntity() != null) {
                    dto.setHttpStatusCode(statusCode);
                    dto.setSuccess(true);
                }

                return dto;
            };

            return client.execute(request, responseHandler);
        } catch (IOException e) {
            LOGGER.error(e.getMessage());
            throw e;
        }
    }

    public ObjectStorageDto get(Tenant tenant, NameSpace nameSpace, String filePath) throws IOException {
        String url = getAddress(tenant, nameSpace, filePath);
        HttpGet request = new HttpGet(url);
        setAuthToken(nameSpace, tenant, (ClassicHttpRequest)request);

        try (CloseableHttpClient client = HttpClients.createDefault()) {
            HttpClientResponseHandler<ObjectStorageDto> responseHandler = response -> {
                int statusCode = response.getCode();
```

```
                        if (statusCode >= 300) {
                                String body = response.getEntity() == null ? "" :
EntityUtils.toString(response.getEntity());
                                throw new IOException("(GET) Failed. status=" + statusCode + "
body=" + body);
                        }

                        ObjectStorageDto dto = new ObjectStorageDto();

                        if (response.getEntity() != null) {
                                dto.setInputSteram(response.getEntity().getContent());
                                dto.setHttpStatusCode(statusCode);
                                dto.setSuccess(true);
                        }

                        return dto;
                };

                return client.execute(request, responseHandler);
        } catch (IOException e) {
                LOGGER.error(e.getMessage());
                throw e;
        }
    }

    public ObjectStorageDto delete(Tenant tenant, NameSpace nameSpace, String filePath) throws
IOException {
        String url = getAddress(tenant, nameSpace, filePath);
        HttpDelete request = new HttpDelete(url);
        setAuthToken(nameSpace, tenant, (ClassicHttpRequest)request);

        try (CloseableHttpClient client = HttpClients.createDefault()) {
                HttpClientResponseHandler<ObjectStorageDto> responseHandler = response -> {
                        int statusCode = response.getCode();

                        if (statusCode >= 300) {
                                String body = response.getEntity() == null ? "" :
EntityUtils.toString(response.getEntity());
                                throw new IOException("(DELETE) Failed. status=" + statusCode +
" body=" + body);
                        }

                        ObjectStorageDto dto = new ObjectStorageDto();

                        if (response.getEntity() != null) {
                                dto.setInputSteram(response.getEntity().getContent());
                                dto.setHttpStatusCode(statusCode);
                                dto.setSuccess(true);
                        }

                        return dto;
                };

                return client.execute(request, responseHandler);
        } catch (IOException e) {
                LOGGER.error(e.getMessage());
                throw e;
        }
    }

    private RequestConfig getRequestConfig() {
```

```
                return RequestConfig.custom()
                                .setConnectionRequestTimeout((connectTimeout <= 0) ? -1 :
(connectTimeout * 1000), TimeUnit.MICROSECONDS)
                                .setResponseTimeout((responseTimeout <= 0) ? -1 : (responseTimeout *
1000), TimeUnit.MILLISECONDS)
                                .build();
        }

        private void setAuthToken(NameSpace nameSpace, Tenant tenant, ClassicHttpRequest request) {
                String token = SpringContext.getProperty("storage.fos.token." + tenant.getTenant() + DOT +
nameSpace.getNameSpace());
                if (LOGGER.isDebugEnabled()) {
                        LOGGER.debug("[FOS - Token: {}]", token);
                }
                if (StringUtils.hasLength(token))
                        request.addHeader(AUTHORIZATION, token);
        }

        private String getAddress(Tenant tenant, NameSpace nameSpace, String filePath) {
                String httpProtocol = SpringContext.getProperty("storage.fos.base-http-protocol");
                String domain = SpringContext.getProperty("storage.fos.base-domain");
                return String.valueOf(httpProtocol) + "://" + nameSpace.getNameSpace() + DOT +
tenant.getTenant() + DOT
                                + (domain.endsWith(SLASH) ? domain : (String.valueOf(domain) + SLASH))
+ "rest"
                                + (filePath.startsWith(SLASH) ? filePath : (SLASH + filePath));
        }

}
package nhnis.fw.commons.fos;

import java.io.InputStream;

import org.springframework.http.MediaType;

import lombok.Getter;
import lombok.Setter;

@Getter
@Setter
public class ObjectStorageDto {

        private boolean isSuccess = false;
        private int httpStatusCode = -1;
        private InputStream inputSteram;
        private MediaType contentType;

}
package nhnis.fw.commons.fos.enums;

public enum Tenant {

        SHARE_OBSTENANT02("obstenant01"),
        BIZ_OBSTENANT02("obstenant02");

        private final String TENANT;

        Tenant(String tenant) {
                this.TENANT = tenant;
        }
```

```
        public String getTenant() {
                return this.TENANT;
        }

}
```

```
package nhnis.fw.commons.fos.enums;

public enum NameSpace {

        // FOS
        OBS_DAY(""),
        OBS_3DAY(""),
        OBS_WEEK(""),
        OBS_2WEEK(""),
        OBS_MON(""),
        OBS_BUNGI(""),
        OBS_BANGI(""),
        OBS_YEAR(""),
        OBS_11YEAR(""),
        OBS_COMM_DAY(""),
        OBS_COMM_3DAY("");

        private final String NAMESPACE;

        NameSpace(String nameSpace) {
                this.NAMESPACE = nameSpace;
        }

        public String getNameSpace() {
                return this.NAMESPACE;
        }
}
```



```
/*****************************************************************************
 * 이 력 사 항
 *****************************************************************************
 * 일 자 버 전 작성자 변경사항
 *****************************************************************************
 * 2026. 7. 6. 1.0 CS549257 최초작성
 *****************************************************************************/
package nhnis.fw.commons.imagelog;

import org.springframework.stereotype.Component;

import nhnis.fw.commons.dto.header.hdr_nhnis;

/**
* <PRE>
```

```
 * 이미지로그 핸들러
 * </PRE>
 *
 * @author CS549257
 * @version 1.0, 2026. 7. 6.
 * @logicalName
 */
@Component
public class ImageLogHandler {

	public void preImagelog(hdr_nhnis header, Object inputDto) {

	}

	public void postImagelog(hdr_nhnis header, Object inputDto) {

	}

	public void exceptionImagelog(hdr_nhnis header, Object inputDto, Throwable t) {

	}

	private void insertImagelog(hdr_nhnis header, Object inputDto, Throwable t) {
		if (t != null) {


		} else {


		}

	}
}
/*****************************************************************************
 * Copyright 2026 by Nonghyup. All rights reserved. Nonghyup 의 사전 승인 없이
 * 본 내용의 전부 또는 일부에 대한 복사, 배포, 사용을 금합니다. Nonghyup의 사전 승인
 * 없이 소스코드를 변경하여 사용하는 경우 소스코드에 대한 품질과 성능을 보장하지 않습니다.
 *
 * If you modify this source without Nonghyup's approval. Nonghyup does
 * not guarantee the quality and performance of source.
 *****************************************************************************/
/*****************************************************************************
* 이 력 사 항
*****************************************************************************
* 일 자 버 전 작성자 변경사항
*****************************************************************************
* 2026. 7. 3. 1.0 CS549257 최초작성
*****************************************************************************/
```